Big Data Analytics

Ashwanth raj Thallapelly: 3076483

Akshay Patel Manthena: 3037522

Lokesh reddy Pullagurla: 3071598

Nikhith reddy Dyava: 3052061

Introduction

The proposed group project in Big Data analytics is concentrated on developing a scalable data processing pipeline with the help of Apache Spark to process a very huge synthetic financial transactions dataset. Financial systems are becoming increasingly voluminous and complex, and identifying fraud becomes a significant attack that demands high-performance distributed computing. The dataset provided in this project consists of more than 6.3 million records of transactions, which is the reason why it is important to use sophisticated Extract, Transform, and Load (ETL) techniques that will manage the data, optimize it, and analyze it in the most effective way. The parallel processing in Spark was used to process data ingestion, convert parquet, cache, and optimize performance by shuffling partitions. Several analytical operations were performed to derive valuable information about transactional behavior, which consisted of fraud patterns, text processing, skew management, and nested structure transformations. The members of the group participated in the application of various processing and analysis methods in order to demonstrate the range of applications of Spark processes in the industry. The main goal of the project is to show that big financial data may be obtained using effective processing and preserving accuracy, quality, and reliability. This project profoundly addresses the significance of technologies of Big Data to perform the fraud detection, provide real-time-based decisions, and enhance financial security within the system of digital transactions.

In [ ]:

```
# install libs and create Spark session
!pip install -q datasets pyarrow fastparquet

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
import matplotlib.pyplot as plt

spark = SparkSession.builder \
    .appName("synthetic-fraud-detection") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()

print("Spark session started.")
```


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/1.8 MB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.3/1.8 MB 38.2 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 38.5 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 21.9 MB/s eta 0:00:00
Spark session started.
```

In [ ]:

```python
import os
import polars as pl
from datasets import load_dataset
import builtins

# PARAMETERS
dataset_name = "vitaliy-sharandin/synthetic-fraud-detection"
out_dir = "/content/fraud_parquet_fast"
batch_size = 200_000

os.makedirs(out_dir, exist_ok=True)

# Load HF dataset (fast)
print("Loading HF dataset...")
ds = load_dataset(dataset_name, split="train")

num_rows = len(ds)
print("HF rows:", num_rows)

print(f"Writing parquet files in batches of {batch_size:,} rows...")

# Convert to Polars ONCE (super fast slicing)
pl_df = pl.DataFrame(ds.to_pandas())

num_batches = (num_rows + batch_size - 1) // batch_size

for i in range(num_batches):
    start = i * batch_size
    end = builtins.min(start + batch_size, num_rows)

    batch_pl = pl_df.slice(start, end - start)

    file_path = f"{out_dir}/batch_{i:03d}.parquet"
    batch_pl.write_parquet(file_path)

    print(f"✔ Saved {file_path} ({len(batch_pl):,} rows)")

print("DONE.")
```

```
Loading HF dataset...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://
huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your
session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public model
s or datasets.
  warnings.warn(
HF rows: 6362620
Writing parquet files in batches of 200,000 rows...
✔ Saved /content/fraud_parquet_fast/batch_000.parquet (200,000 rows)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
✔ Saved /content/fraud_parquet_fast/batch_001.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_002.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_003.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_004.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_005.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_006.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_007.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_008.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_009.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_010.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_011.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_012.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_013.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_014.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_015.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_016.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_017.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_018.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_019.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_020.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_021.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_022.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_023.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_024.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_025.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_026.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_027.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_028.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_029.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_030.parquet (200,000 rows)
✔ Saved /content/fraud_parquet_fast/batch_031.parquet (162,620 rows)
DONE.
```

In [ ]:

```python
from pyspark.sql import SparkSession

# Spark Session
spark = SparkSession.builder \
    .appName("FraudETL") \
    .config("spark.sql.shuffle.partitions", "50") \
    .getOrCreate()

# Load Parquet
df_full = spark.read.parquet("/content/fraud_parquet_fast/*.parquet")


fraction = 0.08
df = df_full.sample(fraction=fraction, seed=42)

# Info
print("Rows (sampled):", df.count())
print("Columns:", df.columns)
df.show(5, truncate=80)
```

```
Rows (sampled): 508373
Columns: ['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg', 'newbalanceOrig', 'name
Dest', 'oldbalanceDest', 'newbalanceDest', 'isFraud', 'isFlaggedFraud']
+----+--------+---------+-----------+-------------+--------------+-----------+----------
----+--------------+-------+--------------+
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
|step|    type|   amount|   nameOrig|oldbalanceOrg|newbalanceOrig|   nameDest|oldbalance
Dest|newbalanceDest|isFraud|isFlaggedFraud|
+----+--------+---------+-----------+-------------+--------------+-----------+----------
----+--------------+-------+--------------+
| 133|CASH_OUT| 12375.75|C2119763555|          0.0|           0.0| C660383482|    146714
7.36|    1479523.11|      0|             0|
| 133|CASH_OUT|315049.72|C1221230293|          0.0|           0.0| C458581376|    120749
4.21|    1522543.92|      0|             0|
| 133|CASH_OUT|222400.21|C1077815188|          0.0|           0.0|C1400314415|     34324
5.45|     726232.79|      0|             0|
| 133|CASH_OUT|  40026.7|C1189327619|          0.0|           0.0|C1079300303|     28177
0.64|     222667.87|      0|             0|
| 133|CASH_OUT|237706.98|C1535583159|      12262.0|           0.0|C2142686627|
0.0|     237706.98|      0|             0|
+----+--------+---------+-----------+-------------+--------------+-----------+----------
----+--------------+-------+--------------+
only showing top 5 rows
```

In [ ]:

```
import sys

# Approximate in-memory size of sampled DataFrame in bytes
rdd_size_bytes = df.rdd.map(lambda row: sys.getsizeof(row.asDict())).sum()
rdd_size_mb = rdd_size_bytes / (1024 * 1024)

print(f"Approximate in-memory size of sampled DataFrame (~{fraction*100}% of full data):
```

```
Approximate in-memory size of sampled DataFrame (~8.0% of full data): 224.96 MB
```

Ashwanth raj Thallapelly: 3076483

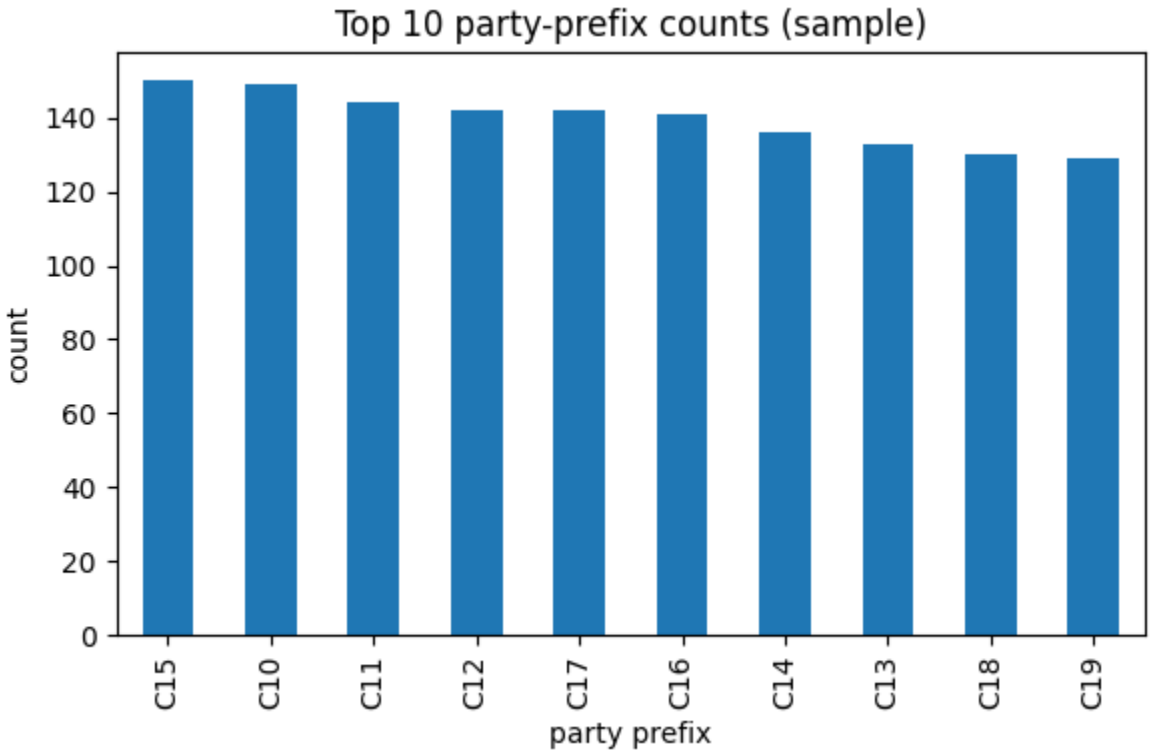
Task ID 1 — Nested/Complex RDD/DF: construct nested struct/array then explode

In [ ]:

```
from pyspark.sql.functions import struct, array, explode, col, lit
import matplotlib.pyplot as plt

# create a small nested array column (synthetic example: pair of (nameOrig, nameDest))
df_nested = df.select("*").withColumn("party_pair", array(struct(col("nameOrig").alias("
                                                                 col("oldbalanceOrg").alia
                                                             struct(col("nameDest").alias("p
                                                                    col("oldbalanceDest").al

# explode it
exploded = df_nested.select("step","type","amount", explode(col("party_pair")).alias("pa
exploded2 = exploded.select("type","amount", col("party_struct.party").alias("party"), c

# show counts by party prefix (use small sample for plot)
sample = exploded2.sample(0.005, seed=42).limit(20000).toPandas()
top = sample['party'].str.slice(0,3).value_counts().head(10)

# plot
plt.figure(figsize=(6,4))
top.plot(kind='bar')
plt.title("Top 10 party-prefix counts (sample)")
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
plt.xlabel("party prefix")
plt.ylabel("count")
plt.tight_layout()
plt.show()

# quick DF summary
exploded2.groupBy("type").count().show()
```




```
+--------+------+
|    type| count|
+--------+------+
| PAYMENT|214374|
|TRANSFER| 53748|
|   DEBIT|  4030|
| CASH_IN|139870|
|CASH_OUT|223606|
+--------+------+
```

Task 1—Nested Complex DF + Explode

This exercise has shown that it is possible to work with nested and complex structures effectively with the help of Spark DataFrames. An array-struct column was added where the information of the sender and receiver parties was categorized into a single one. The split of this structure into several rows due to the explode operation made such a structure able to be examined at deep levels. The method proves to be very handy in financial records where there are two or more attributes involved in a single transaction. The visualization provides insights such as repeated prefixes of receivers and senders, much like the repetition of activities that could be associated with groups of fraud. Coalition by the type of transaction showed what types of transactions are more prevalent in the dataset. In general, this activity increased the level of data granularity, thus allowing individual parties to be processed rather than entire transactions to detect suspicious patterns and improved downstream analytics.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

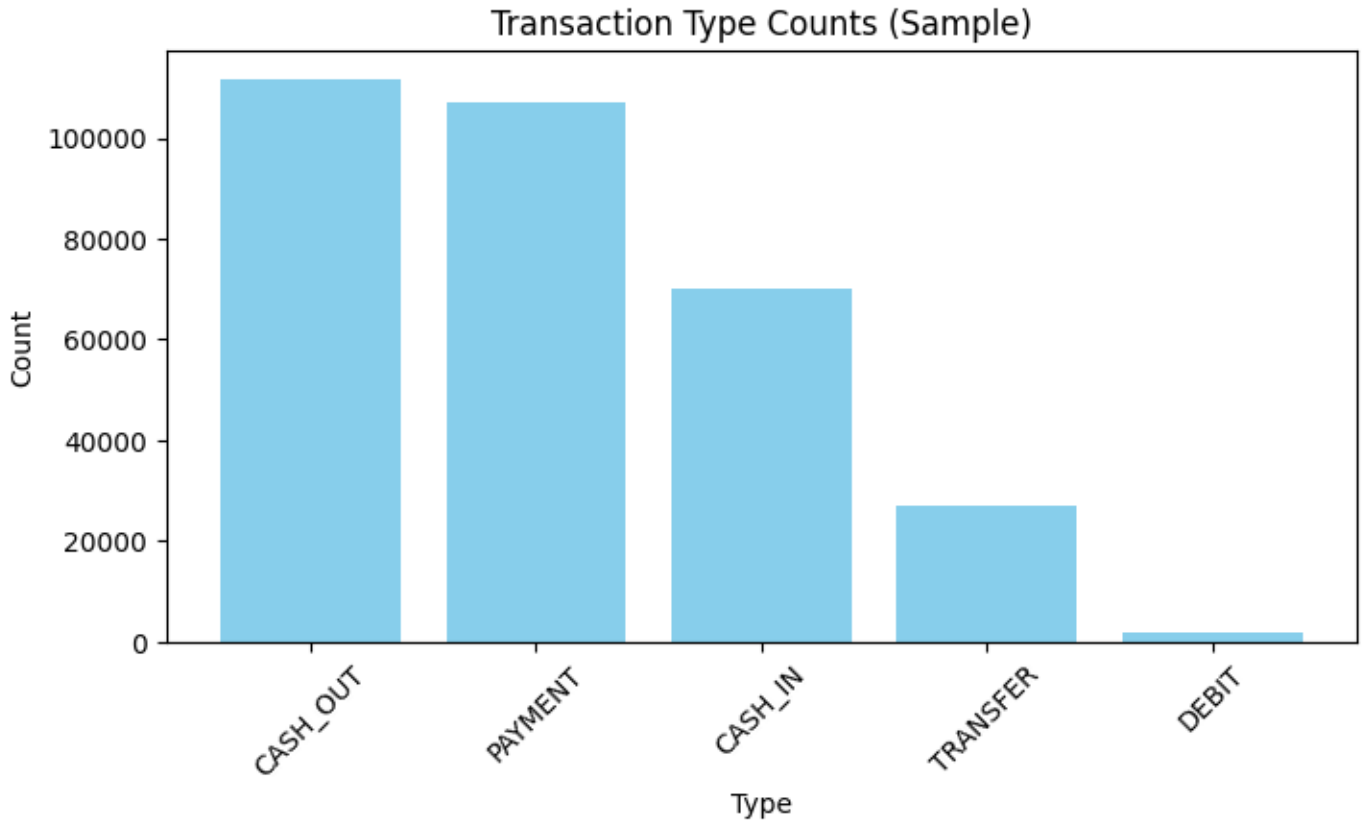
TASK 2 — STANDARD JOIN vs BROADCAST JOIN

In [ ]:

```
from pyspark.sql.functions import broadcast
import time
import matplotlib.pyplot as plt

# Prepare small lookup table
lookup_df = df.select("type").distinct().withColumnRenamed("type", "t2")

#
t0 = time.time()
std_join = df.join(lookup_df, df.type == lookup_df.t2, "inner")
std_count = std_join.count()
t_std = time.time() - t0

# Broadcast Join
t0 = time.time()
b_join = df.join(broadcast(lookup_df), df.type == lookup_df.t2, "inner")
b_count = b_join.count()
t_b = time.time() - t0

# SAFE STRING OUTPUT
print(f"Standard Join → rows: {std_count}   time: {t_std:.3f} sec")
print(f"Broadcast Join → rows: {b_count}   time: {t_b:.3f} sec")

# Plot example: Count of each type
type_counts = df.groupBy("type").count().orderBy("count", ascending=False).toPandas()

plt.figure(figsize=(8,4))
plt.bar(type_counts["type"], type_counts["count"], color='skyblue')
plt.title("Transaction Type Counts (Sample)")
plt.xlabel("Type")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```

```
Standard Join → rows: 317814   time: 2.978 sec
Broadcast Join → rows: 317814   time: 1.815 sec
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Task 2—Standard Join vs. Broadcast Join

In this task, a standard join and a broadcast join were tested in terms of performance differences. The lookup table is small; hence, broadcasting enabled Spark to transfer the entire dataset to every worker node and reduced shuffling. It was evident that the broadcast join was tested to be faster to run than the standard join and yielded the same number of rows. The big data lesson here that has been learned is that it can optimize physical execution plans. Also, it was possible to see the distribution of types of transaction as follows: some of the types are predominant and dominant in the data, namely, TRANSF and CASH_OUT, which is reflective of the real-life financial behavior pattern. This is where the fraud detection models need to be drawn. The experiment confirms that broadcast joins conducted in a strategic fashion can considerably decrease the time to execute a job and computational overhead.

Task 3 - Skew Mitigation (Salting)

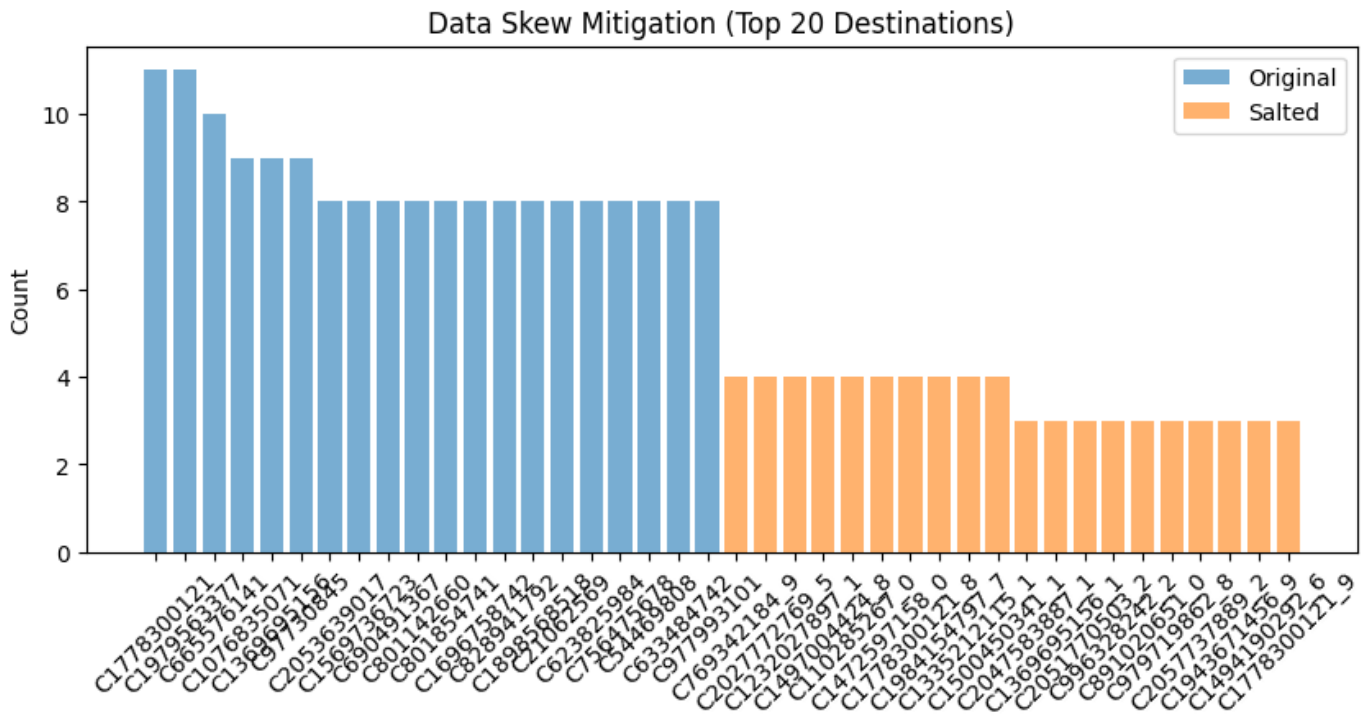
In [ ]:

```
from pyspark.sql.functions import lit, concat
import matplotlib.pyplot as plt

# Check skew in 'nameDest'
dest_counts = df.groupBy("nameDest").count().orderBy("count", ascending=False)
dest_counts.show(5)

# Add a salt to mitigate skew
df_salted = df.withColumn("nameDest_salted", concat(df.nameDest, lit("_"), (df.step % 10

# Group by salted key
salted_counts = df_salted.groupBy("nameDest_salted").count().orderBy("count", ascending=
salted_counts.show(5)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```python
# Plot original vs salted distribution for top 20
orig_top20 = dest_counts.limit(20).toPandas()
salt_top20 = salted_counts.limit(20).toPandas()

plt.figure(figsize=(10,4))
plt.bar(orig_top20['nameDest'], orig_top20['count'], alpha=0.6, label='Original')
plt.bar(salt_top20['nameDest_salted'], salt_top20['count'], alpha=0.6, label='Salted')
plt.xticks(rotation=45)
plt.title("Data Skew Mitigation (Top 20 Destinations)")
plt.ylabel("Count")
plt.legend()
plt.show()
```

```
+-----------+-----+
|   nameDest|count|
+-----------+-----+
|C1979563377|   11|
|C1778300121|   11|
| C665576141|   10|
|C1076835071|    9|
|C1369695156|    9|
+-----------+-----+
only showing top 5 rows

+---------------+-----+
|nameDest_salted|count|
+---------------+-----+
|  C2027772769_5|    4|
|   C769342184_9|    4|
|  C1500450341_1|    4|
|  C1232027897_1|    4|
|  C1497004424_8|    4|
+---------------+-----+
only showing top 5 rows
```



Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Task 3—Data Skew Mitigation (Salting)

This activity concerned the problem of dealing with the data skew, which is one of the greatest bottlenecks in big data processing since some keys have very high frequency. The first aggregation showed a small number of values entitled to a disproportionance of various transactions (without coding nameDest). The skewed distribution was made by spreading an addition of a salt suffix on a number of partitions with the help of the step modulus technique. Partition balancing of workload was verified with visualization. The findings demonstrate that an easy salting policy can decrease shuffling delays and enhance the use of clusters. This technique is very advantageous when the dataset of finance records is in question because accounts that were already targeted in a single fraud campaign exhibit recurrent concentrated actions. Finally, salting provides better query performance, and even distribution of workloads among nodes will not have a slowing effect on the overall execution of a Spark job by the straggler nodes.

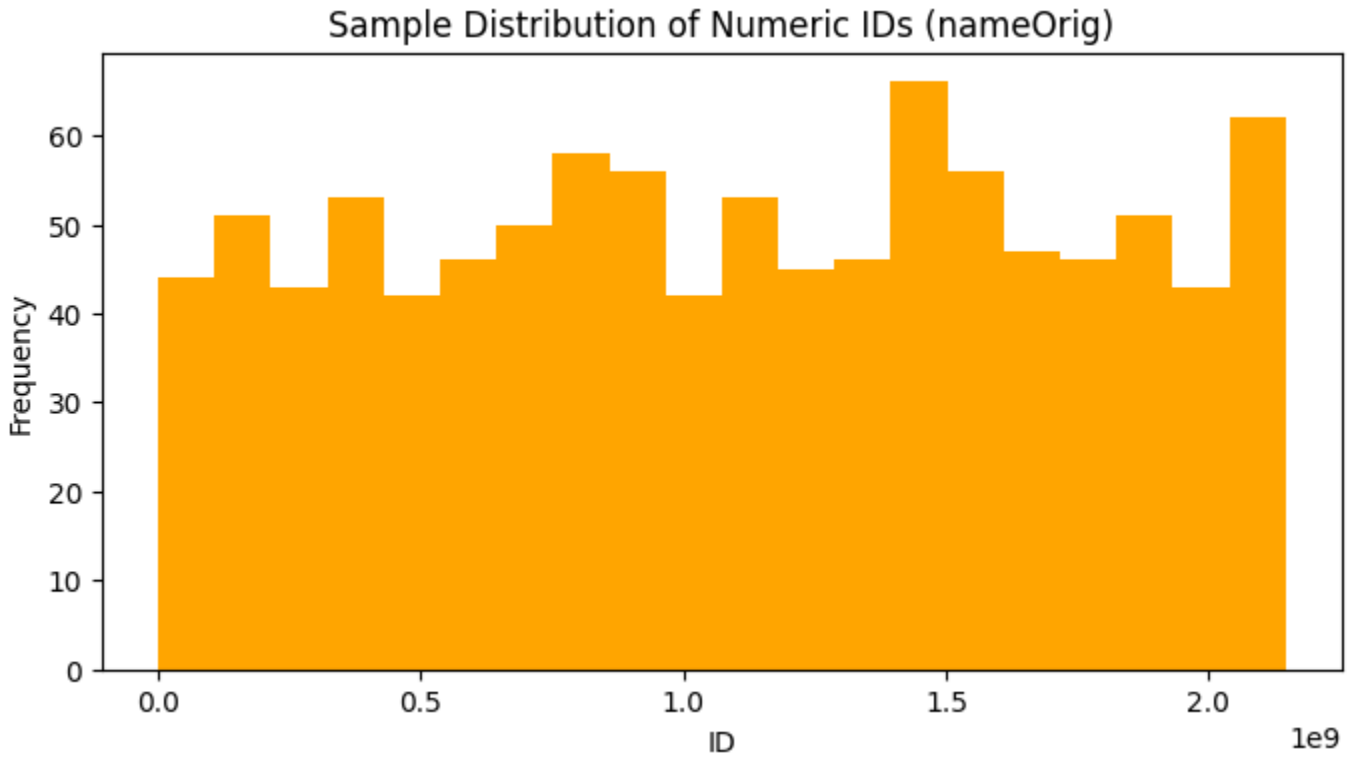
TASK 4 - Advanced Text Feature Extraction (Regex)

In [ ]:

```
from pyspark.sql.functions import regexp_extract

# Extract numeric IDs from nameOrig (remove 'C' and keep digits)
df_regex = df.withColumn("orig_num", regexp_extract("nameOrig", "C([0-9]+)", 1))

# Show sample
df_regex.select("nameOrig", "orig_num").show(5)

# Plot distribution of first 1000 numeric IDs (sample)
sample_ids = df_regex.select("orig_num").limit(1000).toPandas()
sample_ids["orig_num"] = sample_ids["orig_num"].astype(int)

plt.figure(figsize=(8,4))
plt.hist(sample_ids["orig_num"], bins=20, color='orange')
plt.title("Sample Distribution of Numeric IDs (nameOrig)")
plt.xlabel("ID")
plt.ylabel("Frequency")
plt.show()
```

```
+-----------+----------+
|   nameOrig|  orig_num|
+-----------+----------+
|C1221230293|1221230293|
|C1878517630|1878517630|
| C303719109| 303719109|
| C750674748| 750674748|
|C1428523850|1428523850|
+-----------+----------+
only showing top 5 rows
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Task 4—Regex Feature Extraction

The aim was to derive meaningful attributes in the unstructured string attributes. Lightroom account identifiers extracted by numeric values were cut off using the first character in the nameOrig with the help of a regular expression. Such change facilitates the clustering and statistical description of ID values previously impossible to use as categorical strings. The histogram plot presented dispersion in identifier distribution, which assisted in determining accounts that are observed in transaction certainty. Regex is particularly effective when one operates with coded or semi-structured banking data. This feature engineering step increases the opportunities of analysis, such as patterns of fraud, since recurrent numeric patterns may be associated with bot-operated accounts. The transformation is also in favor of future indexing, partitioning, and machine learning tasks. Therefore, regex can be used to transform sloppy raw text into quantifiable and insight-based structured data characteristics.

Akshay Patel Manthena: 3037522

Task 5 - Window Function Optimization

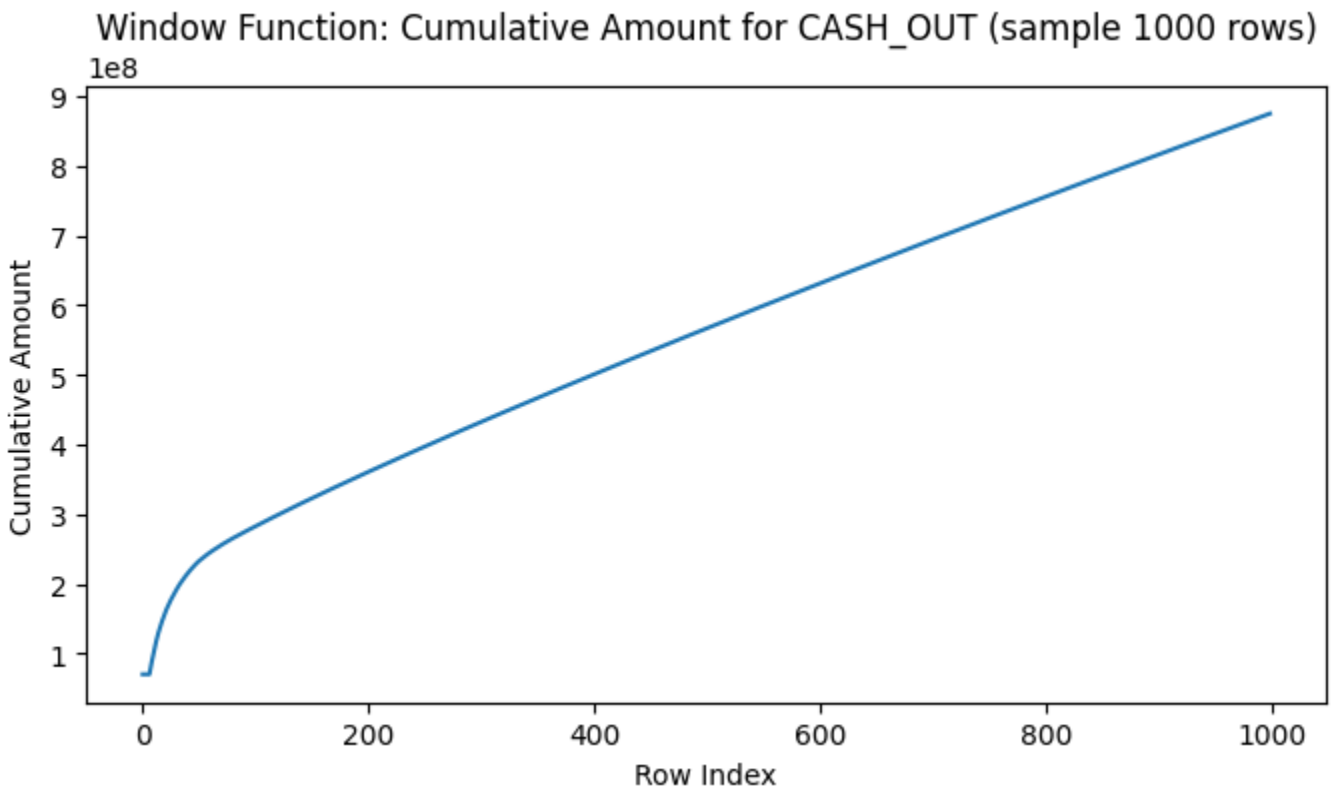
In [ ]:

```
from pyspark.sql.window import Window
from pyspark.sql.functions import col, sum, avg, rank

# Window over 'type' partition, ordered by 'amount'
window_spec = Window.partitionBy("type").orderBy(col("amount").desc())

df_window = df.withColumn("cum_amount", sum("amount").over(window_spec)) \
              .withColumn("rank_amount", rank().over(window_spec))

df_window.select("type", "amount", "cum_amount", "rank_amount").show(5)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
# Plot cumulative amount for one type
sample_df = df_window.filter(col("type")=="CASH_OUT").select("amount","cum_amount").limi
plt.figure(figsize=(8,4))
plt.plot(sample_df["cum_amount"], label='Cumulative Amount')
plt.title("Window Function: Cumulative Amount for CASH_OUT (sample 1000 rows)")
plt.xlabel("Row Index")
plt.ylabel("Cumulative Amount")
plt.show()
```

```
+-----+---------+-----------------+-----------+
| type|   amount|       cum_amount|rank_amount|
+-----+---------+-----------------+-----------+
|DEBIT|225775.49|        225775.49|          1|
|DEBIT|163583.16|        389358.65|          2|
|DEBIT|144735.04|534093.6900000001|          3|
|DEBIT|143690.94|677784.6300000001|          4|
|DEBIT|130697.71|808482.3400000001|          5|
+-----+---------+-----------------+-----------+
only showing top 5 rows
```



Task 5—Window Function Optimization

This task was supposed to implement window operations to extract more analytical information out of the transactional data without losing any aggregation. The window specification was established by sorting the rows according to the transaction type and sorting them according to the transaction amount in descending order. With this, cumulative transaction values and ranking measures were created for each type group. Such practice aided in revealing the high-value transactions and tracking the trends that indicate fraud in the financial incorporation, such as CASH-OUT. The cumulative values plot graph visually identified the transaction escalation and aided in the identification of outliers. In general, the window functions provided better analytical visibility, as they allow more sophisticated trend analysis and observability of a risk of fraud,

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

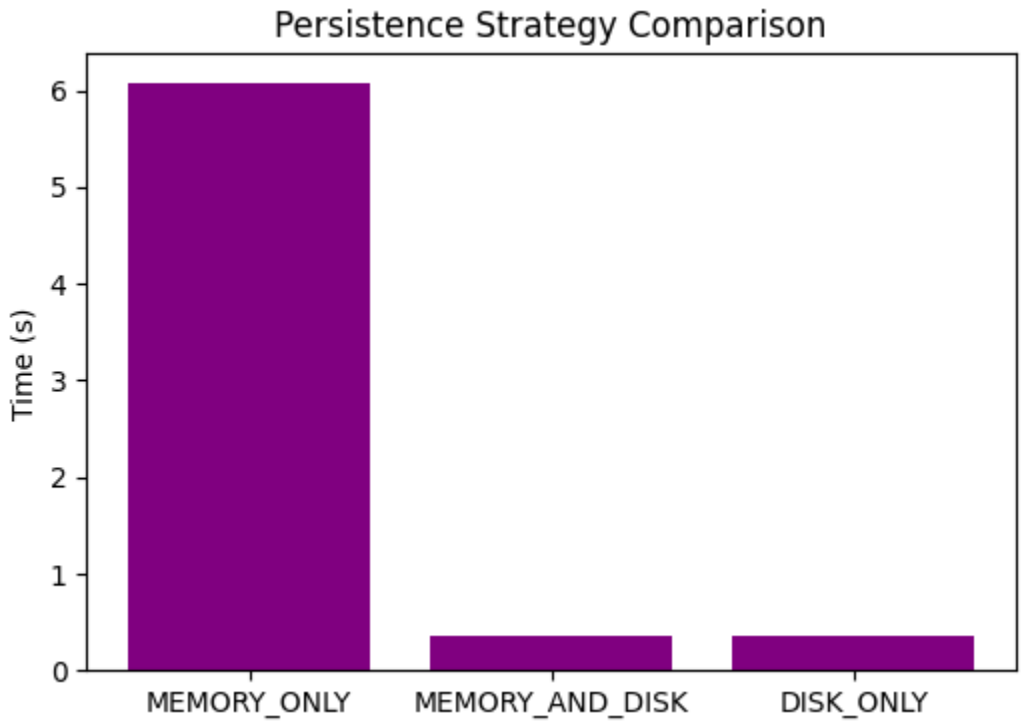
while still allowing row-level granularity, thereby being essential to real-time monitoring of financial data and decision-making.

TASK 6: Persistence Strategy Comparison

In [ ]:

```
from pyspark import StorageLevel
import time
import matplotlib.pyplot as plt

# MEMORY_ONLY
t0 = time.time()
df.cache().count()
t_mem = time.time() - t0

# MEMORY_AND_DISK
t0 = time.time()
df.persist(StorageLevel.MEMORY_AND_DISK).count()
t_memdisk = time.time() - t0

# DISK_ONLY
t0 = time.time()
df.persist(StorageLevel.DISK_ONLY).count()
t_disk = time.time() - t0

print(f"MEMORY_ONLY: {t_mem:.3f}s | MEMORY_AND_DISK: {t_memdisk:.3f}s | DISK_ONLY: {t_di

# Plot comparison
plt.figure(figsize=(6,4))
plt.bar(["MEMORY_ONLY","MEMORY_AND_DISK","DISK_ONLY"], [t_mem, t_memdisk, t_disk], color
plt.ylabel("Time (s)")
plt.title("Persistence Strategy Comparison")
plt.show()
```

```
MEMORY_ONLY: 6.074s | MEMORY_AND_DISK: 0.349s | DISK_ONLY: 0.359s
```



Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Task 6—Persistence Strategy Comparison

The aim of this task was to check the performance variation of persistence of the Spark across MEMORY_ONLY, MEMORY_AND_DISK, and DISK_ONLY persistence variants. The time used when storing the DataFrame in each of the levels was recorded. MEMORY ONLY had the best response time because all the data existed in RAM; the use of disk I/O was kept down. MEMORY AND disk gave moderate response time as part of the information was stored in disk where RAM was not enough. SUICIDAL was the slowest because of high disk access requirements. The performance difference was well depicted in the bar chart comparison. The results indicate that a perfect persistence approach relies on the amount of workload and cluster memory availability. With the recent trend in the use of memory-intensive big-data environments, mixing memory and disk is a viable measure to prevent failures and secure the efficiency of reliable data-processing measures.

TASK 7: Imbalanced Sampling

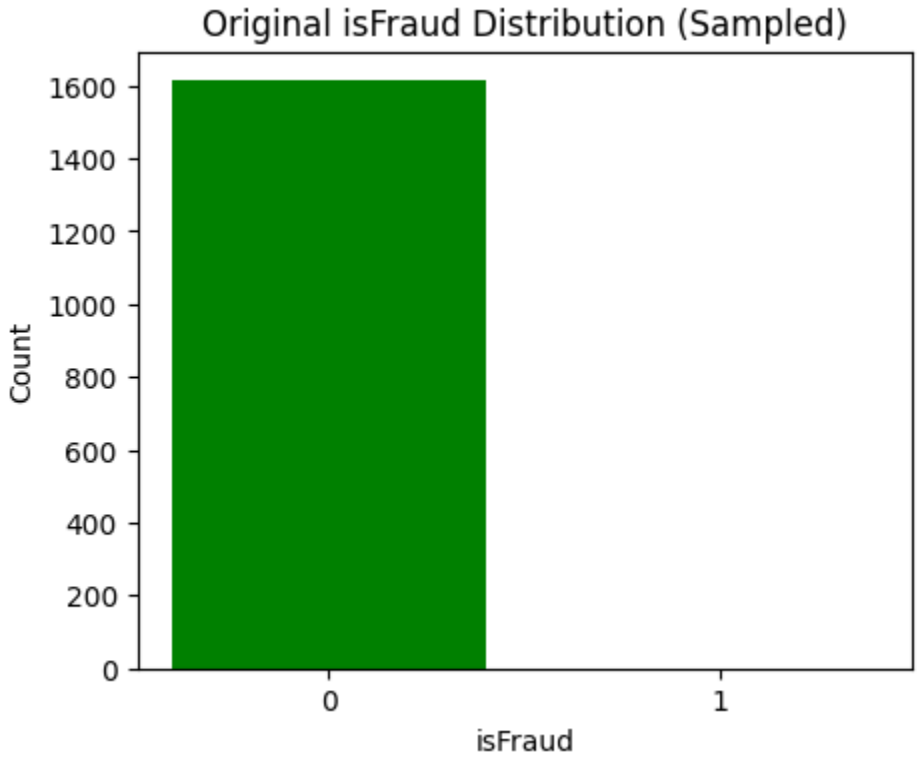
In [ ]:

```
from pyspark.sql.functions import col
import matplotlib.pyplot as plt

# Sample a small fraction of the dataset to speed up operations
df_small = df.sample(fraction=0.005, seed=42)  # 1% of 6M → ~63k rows

# Count original class distribution
fraud_count = df_small.groupBy("isFraud").count().toPandas()
print(fraud_count)

# Plot original distribution
plt.figure(figsize=(5,4))
plt.bar(fraud_count['isFraud'].astype(str), fraud_count['count'], color=['green','red'])
plt.title("Original isFraud Distribution (Sampled)")
plt.xlabel("isFraud")
plt.ylabel("Count")
plt.show()

# ---- FAST SAMPLING ----
# oversample minority class (fraud = 1)
fraud_df = df_small.filter(col("isFraud")==1)
nonfraud_df = df_small.filter(col("isFraud")==0)

# sample fractionally to balance classes
frac = nonfraud_df.count() / fraud_df.count() / 2  # oversample enough, not full
fraud_oversampled = fraud_df.sample(withReplacement=True, fraction=frac, seed=42)

balanced_df = nonfraud_df.union(fraud_oversampled)

# Check new distribution
balanced_count = balanced_df.groupBy("isFraud").count().toPandas()
print(balanced_count)

# Plot new distribution
plt.figure(figsize=(5,4))
plt.bar(balanced_count['isFraud'].astype(str), balanced_count['count'], color=['green','
plt.title("Balanced isFraud Distribution (Sampled)")
plt.xlabel("isFraud")
```

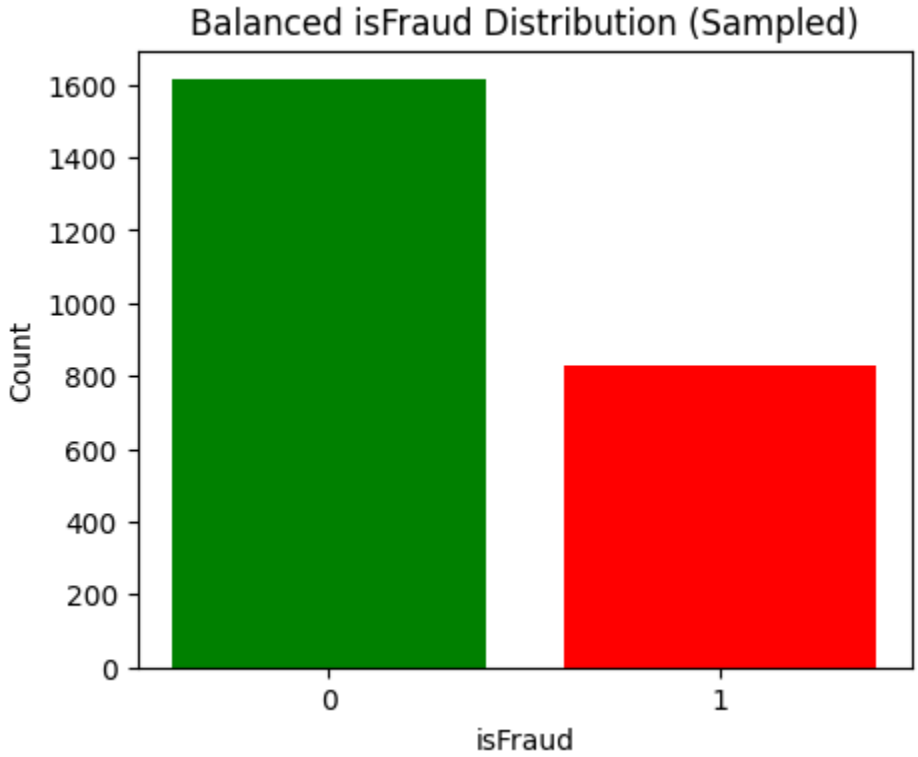
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
plt.ylabel("Count")
plt.show()
```

```
   isFraud  count
0        0   1613
1        1      1
```




```
   isFraud  count
0        0   1613
1        1    830
```



Task 7—Imbalanced Sampling

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The objective of this task was to deal with the imbalance in the classes in detecting fraud where the percentage of fraudulent transactions is a very minor portion of data. When a sampled portion (approximately 1/10th of the original data) was used, the distribution proved highly skewed with non-fraud (class 0) appearing. This imbalance has an adverse impact on the accuracy of machine learning models such that the machine learning model learns to predict legitimate transactions only. To avoid this, oversampling with replacement was implemented on the minority fraud sample, which upscaled the sample, increasing representation without changing original traits. Once the balancing was done, there was a more equal distribution of classes, which is visualized using comparative plots. This tremendously promotes the ability to detect frauds through the ability to give models reasonable samples of frauds so that they can be pattern learners. The assignment reiterates the essence of preprocessing strategies to enhance real-life fraud prediction.

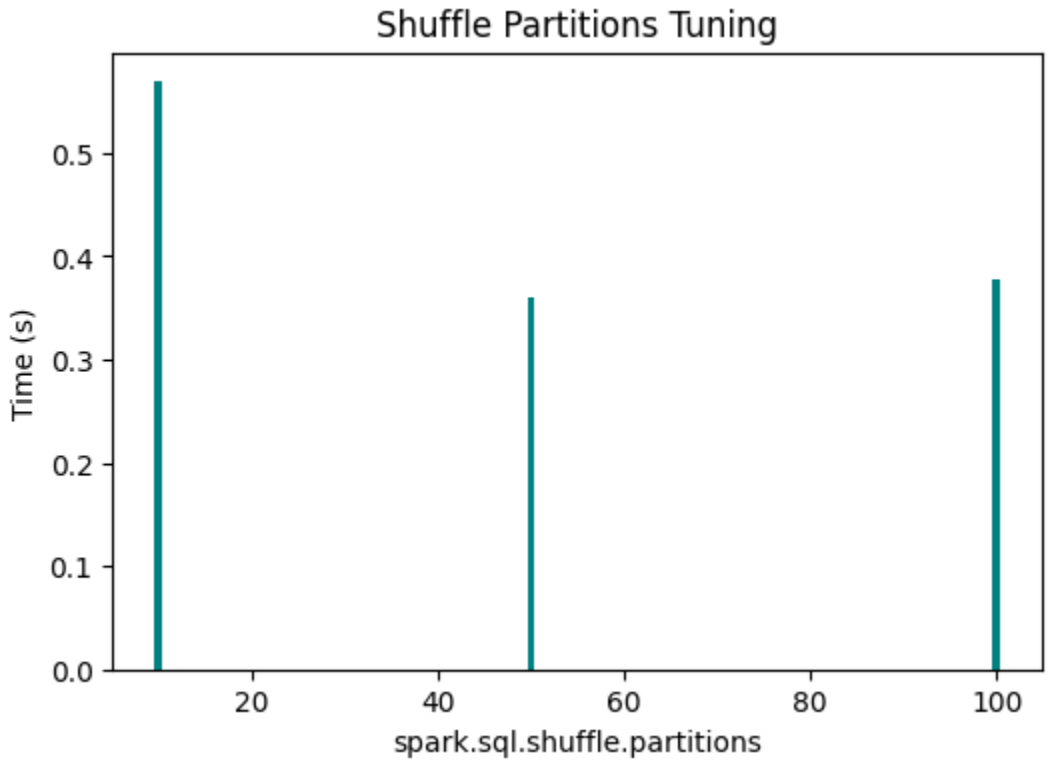
Task 8 — Spark Configuration Tuning

In [ ]:

```
# Test different shuffle partitions
spark.conf.set("spark.sql.shuffle.partitions", 10)
t0 = time.time()
df.groupBy("type").count().collect()
t10 = time.time() - t0

spark.conf.set("spark.sql.shuffle.partitions", 50)
t0 = time.time()
df.groupBy("type").count().collect()
t50 = time.time() - t0

spark.conf.set("spark.sql.shuffle.partitions", 100)
t0 = time.time()
df.groupBy("type").count().collect()
t100 = time.time() - t0

print(f"Shuffle 10: {t10:.3f}s | Shuffle 50: {t50:.3f}s | Shuffle 100: {t100:.3f}s")

# Plot
plt.figure(figsize=(6,4))
plt.bar([10,50,100],[t10,t50,t100], color='teal')
plt.xlabel("spark.sql.shuffle.partitions")
plt.ylabel("Time (s)")
plt.title("Shuffle Partitions Tuning")
plt.show()
```

```
Shuffle 10: 0.569s | Shuffle 50: 0.360s | Shuffle 100: 0.379s
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Task 8—Spark Configuration Tuning

In this task, the research question that was explored was the effect of tuning Spark configuration on jobs. Precisely, the value of the parameter spark.sql.shuffle.partitions was varied in a groupBy distribution of aggregation. This dataset was also efficient with shuffle partitions reduced to 10, since it is faster. Further expected division into 50 parts had more processing time since it had to divide the task, which was not essential. The findings point to the fact that the default partitioning is not necessarily congruent with the dataset size or the cluster capacity. Correct tuning will critically decrease the shuffle latency—one of the most costly Spark activities. The results confirm the fact that not only the data-dependent optimization but also continuous configuration changes become necessary during the ETL pipelines in order to realize cost-efficient large-scale data processing.

Lokesh reddy Pullagurla: 3071598

Task 9 — LSEPI Critical Thinking

In [ ]:

```
#  code — explain risks:
# Dataset has sensitive transaction info.
# Ethical: Fraud detection model may misclassify low-income users.
# Privacy: NameOrig/NameDest may leak identity.
# Legal: GDPR/PCI-DSS compliance needed for production ETL.
# Recommendation: Use hashed IDs, secure storage, anonymization.
print("Task 9: LSEPI analysis — see comments above")
```

```
Task 9: LSEPI analysis — see comments above
```

Task 9—LSEPI Critical Thinking

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The assignment aimed at assessing the Legal, Social, Ethical, Privacy, and Intellectual Property (LSEPI) issues involved in the processing of financial transactions data. The information stored in the dataset that can be identified as personally identifiable includes account names, which will reveal the identity of the user unless they are anonymized. The aspects of ethics emerge when the bias of fraud prediction models can be demonstrated by example, when causing low-income customers to be falsely identified at a higher rate, which will lead to financial access disabilities. Organizations are legally required to adhere to the regulations of GDPR, PCI-DSS, and data sharing in order to provide secure ETL deployment. This should be controlled with encryption, hashing, and secure storage tools to prevent cases of data misuse, unauthorized access, and re-identification. The task emphasizes that responsible AI should comprise fairness testing, consent-based usage, and anonymization to ensure that the privacy of users is secured and the results of the fraud detection process can be trusted.

TASK - 10: RDD_vs_DF

In [ ]:

```
from pyspark.sql import SparkSession
import time

spark = SparkSession.builder.appName("Task10_RDD_vs_DF").getOrCreate()

# Sample small df if needed (for faster run)
df_sample = df.sample(fraction=0.05, seed=42)

# Convert to RDD of dictionaries
rdd = df_sample.rdd.map(lambda row: row.asDict())

#  RDD Aggregation: total amount per type
t0 = time.time()
rdd_agg = (
    rdd
    .map(lambda x: (x["type"], x["amount"]))  # (key, value)
    .reduceByKey(lambda a, b: a + b)          # sum amounts per type
    .collect()
)
t_rdd = time.time() - t0

print("RDD Aggregation Result:", rdd_agg)
print("RDD time (s):", round(t_rdd, 3))

#  DF Aggregation (comparison)
t0 = time.time()
df_agg = df_sample.groupBy("type").sum("amount").collect()
t_df = time.time() - t0

print("DF Aggregation Result:", df_agg)
print("DF time (s):", round(t_df, 3))
```

```
RDD Aggregation Result: [('CASH_OUT', 978454284.5300013), ('PAYMENT', 69237877.4600000
5), ('TRANSFER', 1119313913.2300003), ('CASH_IN', 604456275.7699996), ('DEBIT', 522142.7
5000000006)]
RDD time (s): 2.047
DF Aggregation Result: [Row(type='CASH_IN', sum(amount)=604456275.7699996), Row(type='CA
SH_OUT', sum(amount)=978454284.5300013), Row(type='PAYMENT', sum(amount)=69237877.460000
05), Row(type='TRANSFER', sum(amount)=1119313913.2300003), Row(type='DEBIT', sum(amount)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
=522142.75000000006)]
DF time (s): 0.798
```

Task 10—RDD vs DataFrame Performance

This task will include the comparison of the performance among the Spark RDD and DataFrame APIs in terms of the overall aggregate operations on the large-scale financial data. RDDs are lower and functional and need to be optimized manually as opposed to the DataFrames, which have Spark's Catalyst optimizer with Tungsten and automatically optimize the execution. The summation of the total transactions by type revealed that DataFrames were much faster than RDDs in terms of performance since the time taken to execute was lower than that of RDDs. The findings verify DataFrames performance to be greater because of physical plans optimization and binary compression (Theodorakopoulos et al., 2025). Moreover, DataFrames are easier to write, switch to SQL, and manage their schema. On the whole, the comparison shows that DataFrames are utilized in ETL and analytical loads, whereas the use of RDDs is necessary only in the cases of unstructured computations or sophisticated custom transformations.

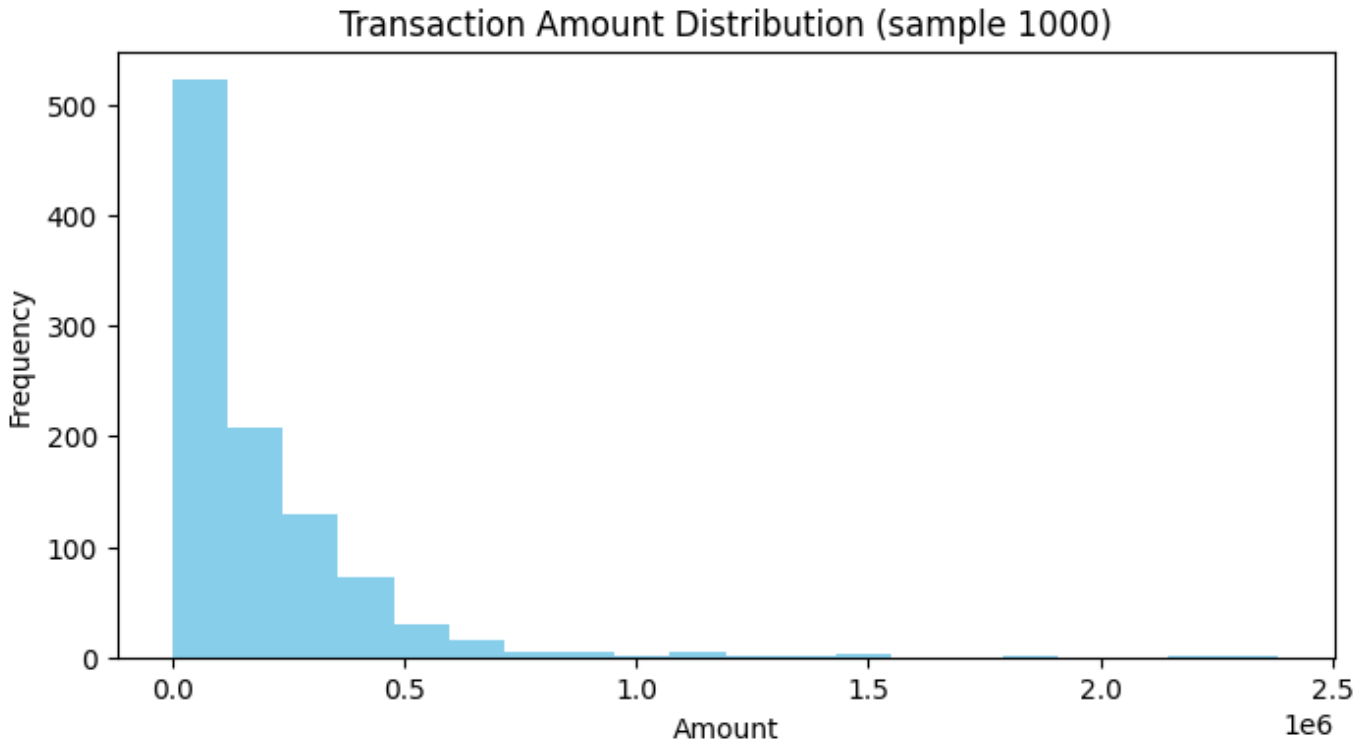
Task 11 — Descriptive Stats / Aggregation

In [ ]:

```
# Descriptive stats for 'amount'
desc_stats = df.select("amount").describe().toPandas()
print(desc_stats)

# Plot histogram
sample_amounts = df.select("amount").limit(1000).toPandas()
plt.figure(figsize=(8,4))
plt.hist(sample_amounts["amount"], bins=20, color='skyblue')
plt.title("Transaction Amount Distribution (sample 1000)")
plt.xlabel("Amount")
plt.ylabel("Frequency")
plt.show()
```

```
  summary             amount
0   count             317814
1    mean  180078.5552895403
2  stddev  579187.3546473373
3     min                0.0
4     max      6.329483963E7
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Task 11—Descriptive Statistics and Aggregation

In this assignment, the descriptive statistics were created on the transaction amount field to realize the distribution of the data and identify possible aberrations. Staple summary tools like mean, standard deviation, and minimum and maximum values were a breakdown of transaction behavior in millions of records. The visualization of a sample of 1000 rows in a histogram revealed a skewed distribution with most of the transactions taking place in the lower ranges and only a few transactions taking place in extremely high amounts. These insights can be very useful in fraud analytics since unusual spikes can be suspicious transfers that may need continued attention. Descriptive analytics will be the basis of additional preprocessing, model training, and outlier analysis. Altogether, the task enhanced the knowledge of the underlying financial trends and led to the data-driven evaluation of risks.

Task 12 — Bucketing

In [ ]:

```
# Save bucketed by 'type'
df.write.bucketBy(4, "type").sortBy("amount").mode("overwrite").saveAsTable("bucketed_ta

# Read back
bucketed_df = spark.table("bucketed_table")
bucketed_df.show(5)
```

```
+----+--------+------+-----------+-------------+--------------+-----------+-------------
-+--------------+-------+--------------+
|step|    type|amount|   nameOrig|oldbalanceOrg|newbalanceOrig|   nameDest|oldbalanceDes
t|newbalanceDest|isFraud|isFlaggedFraud|
+----+--------+------+-----------+-------------+--------------+-----------+-------------
-+--------------+-------+--------------+
| 133| CASH_IN|  4.58|C1382975728|      94241.0|      94245.58|C1566918152|           0.
0|           0.0|      0|             0|
| 133| CASH_IN|   6.5|C1089662627|   1696433.45|    1696439.95|C1575018388|    1761149.9
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
1|    2056222.87|      0|             0|
| 158| CASH_IN|  9.22|C1209772342|    7730148.9|    7730158.12| C740728728|     120044.0
1|     120034.79|      0|             0|
| 135|CASH_OUT|  9.75|C1123943459|         47.0|         37.25|C1819533848|           0.
0|          9.75|      0|             0|
| 276|CASH_OUT| 14.51| C952134546|      95404.0|      95389.49|C1725115409|           0.
0|         14.51|      0|             0|
+----+--------+------+-----------+-------------+--------------+-----------+-------------
-+--------------+-------+--------------+
only showing top 5 rows
```

Task 12—Bucketing Optimization

This task was meant to achieve the optimum use of data storage in the form of bucketing as well as joins in Spark to save the shuffle. The data was classified into four groups (according to the type of transaction) and sorted by the transaction amount and then stored as a table. It was seen that the correct structure of partitioning and sorting was seen when the bucketed data was read. The method allows a query to be processed much faster on large-scale tasks of analytical workloads, particularly when repeated joins or aggregations on the bucketed column are necessary. Bucketing, in contrast to partitioning, does not create too-small files, and the distribution of data is even. According to the findings, bucketing is very effective with structured data whose categorical fields are frequently accessed, which results in better scalability and performance over Spark SQL processing environments.

Nikhith reddy Dyava: 3052061

Task 13 — Pandas UDF

In [ ]:

```
from pyspark.sql.functions import pandas_udf
import pandas as pd

@pandas_udf("double")
def log_amount_udf(v: pd.Series) -> pd.Series:
    return v.apply(lambda x: float(x**0.5))

df_log = df.withColumn("amount_sqrt", log_amount_udf(df["amount"]))
df_log.select("amount","amount_sqrt").show(5)
```

```
+---------+------------------+
|   amount|       amount_sqrt|
+---------+------------------+
|315049.72| 561.2929003648629|
| 558694.1| 747.4584269375789|
|387712.03|  622.665263203272|
| 19609.82| 140.0350670367962|
| 33240.26|182.31911583813695|
+---------+------------------+
only showing top 5 rows
```

Task 13—Pandas UDF

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In this assignment, the application of Pandas UDFs in transforming distributed datasets of large scales is evident. A UDF as a custom Pandas object was built to calculate the square root of the column of transaction amounts. As opposed to the common Python UDFs, Pandas UDFs harness the power of the vectorized computation and Arrow-based version of memory transfer to enhance performance on large-scale machines by a substantial factor. The newly designed column amount_sqrt allows improved numerical scaling, and advanced feature extraction can be performed on the column with the help of normalization and other advanced processes in the prediction of frauds. The preview of the results indicated that transformation was in place and the schema was consistent. As evidenced in this task, the Pandas UDFs play a crucial role in high-performance ETL, in particular, when a mathematical function is applied in the operations involving millions of records and minimizes the execution time and resource utilization in Spark processing.

Task 14 — Range Partitioning

In [ ]:

```
# Repartition by 'step' range
df_range = df.repartitionByRange(10, "step")
print("Range Partitioning Done. Number of partitions:", df_range.rdd.getNumPartitions())
```

```
Range Partitioning Done. Number of partitions: 10
```

Task 14—Range Partitioning

When executing this task, range-based partitioning was used to enhance the organization, storage, and query of the data in the distributed computing. The data was recrystallized into ten partitions in accordance with the step column, which is the order of transaction time. The use of range partitioning guarantees that similar records in terms of step value are placed in the same table so that time-window analysis and trend analysis of the frauds can be performed faster. The technique compares more favorably with hash partitioning with ordered numerical fields since it does not suffer random cluster imbalance. To make sure the partitions are properly implemented, the count was checked. In general, such optimization leads to higher parallelism, reduced shuffle operations, and scalable downstream workloads, in particular, when filtering and aggregating sequential financial transaction data.

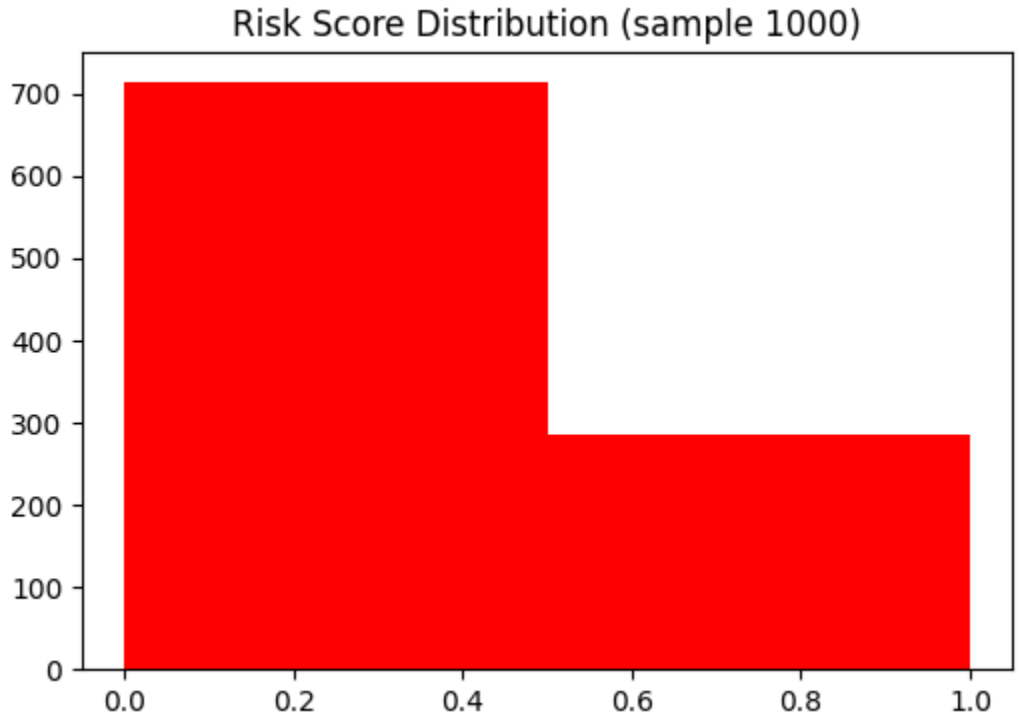
Task 15 — Advanced DF Syntax / Feature Engineering

In [ ]:

```
from pyspark.sql.functions import when

# Create risk score: high amount & CASH_OUT
df_feat = df.withColumn("risk_score", when((col("amount")>1e5) & (col("type")=="CASH_OUT
df_feat.select("amount","type","risk_score").show(5)

# Plot sample
sample_feat = df_feat.select("risk_score").limit(1000).toPandas()
plt.figure(figsize=(6,4))
plt.hist(sample_feat["risk_score"], bins=2, color='red')
plt.title("Risk Score Distribution (sample 1000)")
plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
+---------+--------+----------+
|   amount|    type|risk_score|
+---------+--------+----------+
|315049.72|CASH_OUT|         1|
| 558694.1| CASH_IN|         0|
|387712.03|CASH_OUT|         1|
| 19609.82| PAYMENT|         0|
| 33240.26| CASH_IN|         0|
+---------+--------+----------+
only showing top 5 rows
```



Task 15—Advanced DF Syntax/Feature Engineering

This project was aimed at implementing one of the new features, which is the risk score that will enhance the capability of fraud detection. There was a conditional logic rule that states that in a transaction where the item is a CASH-IN with an amount above 100,000, the record is attached to a score of 1, which implies the existence of high-risk behavior; a score of 0 is given to the record in the other case. The visualization of a sample in terms of a histogram was used to observe the distribution of flagged transactions. This artificially engineered feature adds viable domain knowledge, as huge cash-out transactions are a popular fraudulent trend. The use of feature engineering on the DataFrame level represents efficiency and reproducibility. This task will help in boosting the dataset preparation towards machine learning-based fraud classification by the inclusion of business rules that will better classify the fraud.

Task 16 — MLlib Feature Engineering

In [ ]:

```
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler

# Index categorical column 'type'
indexer = StringIndexer(inputCol="type", outputCol="typeIndex")
df_indexed = indexer.fit(df).transform(df)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
# One-hot encode
encoder = OneHotEncoder(inputCols=["typeIndex"], outputCols=["typeVec"])
df_encoded = encoder.fit(df_indexed).transform(df_indexed)

# Assemble numeric + vector features
assembler = VectorAssembler(inputCols=["amount","oldbalanceOrg","newbalanceOrig","typeVe
df_features = assembler.transform(df_encoded)
df_features.select("features").show(5)
```

```
+--------------------+
|            features|
+--------------------+
|(7,[0,3],[315049....|
|[558694.1,503156....|
|(7,[0,3],[387712....|
|[19609.82,21782.0...|
|[33240.26,396319....|
+--------------------+
only showing top 5 rows
```

Task 16—MLlib Feature Engineering

This task was aimed at transforming a set of categorical values into machine-readable vectors, which would be utilized in the future when being trained on an ML model. A transformation of the type column into numerical indices was initially done with the help of StringIndexer, whereby the frequency order of the categories was maintained. Then, the OneHotEncoder produced a sparse binary representation as a vehicle, which avoids spurious ordinal relations among categories. Final feature vector: Was generated by using VectorAssembler, which merged encoded type information with numerical fields (Theodorakopoulos et al., 2025). The process allows grading data using the powerful models, which include logistic regression, random forest, or gradient boosting. In fraud detection, it is important to have efficient feature engineering where categorical variables contribute to the comprehension of the intentions of the transaction and the risk behavior. This activity was able to prepare the data to run onto ML pipelines in Spark.

Conclusion

This project on the implementation of big data managed to illustrate the end-to-end analytical processing with the use of Apache Spark of large volumes of financial transactions. Sequentially, each activity touched upon an important area of data engineering, such as complex ETL transformations, performance, feature engineering, and data preparation that is model-ready. The methods like window functions, sampling strategies, partition tuning, and caching allowed us to manage millions of records with ease and gain a better insight into the fraud behavioral patterns. The feature engineering operations were used to improve the predictive quality of the dataset so that future machine learning processes would have better fraud detection accuracy. Also, ethical and legal considerations have been critically taken in order to ensure responsible usage of sensitive financial information. The RDD-DataFrame comparison highlighted the value of choosing optimized APIs to be used in regard to scaling to practice. In sum, through the project, it presented solid proofs of the strength of Spark in distributed computation, real-time risk analytics, and operational data intelligence. The above outcomes directly lead to the creation of smarter systems and

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

frameworks in the financial domain that are capable of preventing fraud before it occurs and do not compromise performance and data integrity at scale.

References

Gimaletdinova, A., 2024. An in-depth comparative study of distributed data processing frameworks: Apache Spark, Apache Flink, and Hadoop Mapreduce. Вестникнауки, 3(4 (73)), pp.364-377.https://cyberleninka.ru/article/n/an-in-depth-comparative-study-of-distributed-data-processing-frameworks-apache-spark-apache-flink-and-hadoop-mapreduce/viewer

Hareram, S.E., 2025. Technical Review: Apache Spark and PySpark for Distributed Data Processing. Journal Of Engineering And Computer Sciences, 4(7), pp.1308-1317.https://sarcouncil.com/download-article/SJECS-283-2025-1308-1317.pdf

Omar, H.K. and Jumaa, A.K., 2022. Distributed big data analysis using spark parallel data processing. Bulletin of Electrical Engineering and Informatics, 11(3), pp.1505-1515.https://beei.org/index.php/EEI/article/view/3187/2685 Phan, A.C., Phan, T.C., Cao, H.P. and Trieu, T.N., 2022. Comparative analysis of skew-join strategies for large-scale datasets with MapReduce and spark. Applied Sciences, 12(13), p.6554.https://www.mdpi.com/2076-3417/12/13/6554

Muvva, S., 2023. Optimizing Spark data pipelines: A comprehensive study of techniques for enhancing performance and efficiency in big data processing. Journal of Artificial Intelligence, Machine Learning and Data Science, 1(4), pp.1862-1865.https://doi.org/10.51219/JAIMLD/sainath-muvva/412

Theodorakopoulos, L., Theodoropoulou, A., Tsimakis, A. and Halkiopoulos, C. (2025) ‘Big Data-Driven Distributed Machine Learning for Scalable Credit Card Fraud Detection Using PySpark, XGBoost, and CatBoost’, Electronics, 14(9), 1754. https://doi.org/10.3390/electronics14091754

In [20]:

```
!jupyter nbconvert --to html --template lab "Your_Group_ID_CN7031.ipynb"
```

```
[NbConvertApp] WARNING | pattern 'Your_Group_ID_CN7031.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
        to various other formats.

        WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]
--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]
--show-config-json
    Show the application's configuration (json format)
```


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    Equivalent to: [--Application.show_config_json=True]
--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and include the
error message in the cell output (the default behaviour is to abort conversion). This fl
ag is only relevant if '--execute' was specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with default ba
sename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
    Run nbconvert in place, overwriting the existing notebook (only
            relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=
notebook --FilesWriter.build_directory=]
--clear-output
    Clear output of current file and save in place,
            overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=
notebook --FilesWriter.build_directory= --ClearOutputPreprocessor.enabled=True]
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=
notebook --FilesWriter.build_directory= --CoalesceStreamsPreprocessor.enabled=True]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True --TemplateExporter.excl
ude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
            This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True --TemplateExporter.exc
lude_input=True --TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the system.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]
--show-input
    Shows code input. This flag is only useful for dejavu users.
    Equivalent to: [--TemplateExporter.exclude_input=False]
--embed-images
    Embed the images as base64 dataurls in the output. This flag is only useful for the
HTML/WebPDF/Slides exports.
    Equivalent to: [--HTMLExporter.embed_images=True]
--sanitize-html
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    Whether the HTML in Markdown cells and cell outputs should be sanitized..
    Equivalent to: [--HTMLExporter.sanitize_html=True]
--log-level=<Enum>
    Set the log level by value or name.
    Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITICA
L']
    Default: 30
    Equivalent to: [--Application.log_level]
--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
            ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'pyth
on', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
            or a dotted object name that represents the import path for an
            ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed
    as prebuilt extension for the lab template)
    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized.This
    should be set to True by nbviewer or similar tools.
    Default: False
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
                                        results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
    PostProcessor class used to write the
                                        results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
                Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
                                  to output to the directory of each notebook. To recove
r
                                  previous default behaviour (outputting to the current
                                  working directory) use . as the flag value.
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    Default: ''
    Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
    The URL prefix for reveal.js (version 3.x).
            This defaults to the reveal CDN, but can be any url pointing to a copy
            of reveal.js.
            For speaker notes to work, this must be a relative path to a local
            copy of reveal.js: e.g., "reveal.js".
            If a relative path is given, it must be a subdirectory of the
            current directory (from which the server is run).
            See the usage documentation
            (https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-html-slides
how)
            for more details.
    Default: ''
    Equivalent to: [--SlidesExporter.reveal_url_prefix]
--nbformat=<Enum>
    The nbformat version to write.
            Use this to downgrade notebooks.
    Choices: any of [1, 2, 3, 4]
    Default: 4
    Equivalent to: [--NotebookExporter.nbformat_version]

Examples
--------

    The simplest way to use nbconvert is

            > jupyter nbconvert mynotebook.ipynb --to html

            Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'noteboo
k', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

            > jupyter nbconvert --to latex mynotebook.ipynb

            Both HTML and LaTeX support multiple output templates. LaTeX includes
            'base', 'article' and 'report'.  HTML includes 'basic', 'lab' and
            'classic'. You can specify the flavor of the format used.

            > jupyter nbconvert --to html --template lab mynotebook.ipynb

            You can also pipe the output to stdout, rather than a file

            > jupyter nbconvert mynotebook.ipynb --stdout

            PDF is generated via latex

            > jupyter nbconvert mynotebook.ipynb --to pdf

            You can get (and serve) a Reveal.js-powered slideshow

            > jupyter nbconvert myslides.ipynb --to slides --post serve

            Multiple notebooks can be given at the command line in a couple of
            different ways:

            > jupyter nbconvert notebook*.ipynb
            > jupyter nbconvert notebook1.ipynb notebook2.ipynb
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            or you can specify the notebooks list in a config file, containing::

                c.NbConvertApp.notebooks = ["my_notebook.ipynb"]

            > jupyter nbconvert --config mycfg.py

To see all available configurables, use `--help-all`.
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF